TOPIC 5 : GREEDY

1. There are 3n piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers piles where piles[i] is the number of coins in the ith pile. Return the maximum number of coins that you can have.

Example 1:

Input: piles = [2,4,1,2,7,8]
Output: 9

Explanation: Choose the triplet (2, 7, 8), Alice Pick the pile with 8 coins, you the pile with 7 coins and Bob the last one.

Choose the triplet (1, 2, 4), Alice Pick the pile with 4 coins, you the pile with 2 coins and Bob the last one.

The maximum number of coins which you can have is: 7 + 2 = 9.

On the other hand if we choose this arrangement (1, 2, 8), (2, 4, 7) you only get 2 + 4 = 6 coins which is not optimal.

Example 2:

Input: piles = [2,4,5]

Output: 4

Aim

To find the maximum number of coins you can collect using the greedy approach.

Algorithm

1. Sort the piles in ascending order.
2. Alice always takes the largest pile.
3. You take the second largest pile.
4. Bob takes the smallest pile.
5. Repeat until all piles are used.
6. Add the coins you collected.

Python Code

File Edit Format Run Options Windows Help

```
piles = [2,4,1,2,7,8]

piles.sort()

ans = 0
l = 0
r = len(piles) - 1

while l < r:
    r -= 1
    ans += piles[r]
    r -= 1
    l += 1

print("Maximum coins:", ans)
```

Input

piles = [2,4,1,2,7,8]

Output

```
>>> ================================ RESTART ================================
>>>
('Maximum coins:', 9)
>>>
```

Result

The program uses a greedy strategy to maximize the number of coins collected.

2. You are given a 0-indexed integer array coins, representing the values of the coins available, and an integer target. An integer x is obtainable if there exists a subsequence of coins that sums to x. Return the minimum number of coins of any value that need to be added to the array so that every integer in the range [1, target] is obtainable. A subsequence of an array is a new non-empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining elements.

Example 1:

Input: coins = [1,4,10], target = 19

Output: 2

Explanation: We need to add coins 2 and 8. The resulting array will be [1, 2, 4, 8, 10].

It can be shown that all integers from 1 to 19 are obtainable from the resulting array, and that 2 is the minimum number of coins that need to be added to the array.

Example 2:
Input: coins = [1, 4, 10, 5, 7, 19], target = 19

Output: 1

Explanation: We only need to add the coin 2. The resulting array will be [1,2, 4, 5, 7, 10, 19].

It can be shown that all integers from 1 to 19 are obtainable from the resulting array, and that 1 is the minimum number of coins that need to be added to the array .

Aim

To find the minimum number of coins required so that every value from 1 to target can be formed.

Algorithm

1. Sort the coins.
2. Maintain the smallest value that cannot be formed (reach).
3. If the current coin is less than or equal to reach, extend the reachable range.
4. Otherwise, add a new coin equal to reach.
5. Repeat until all values up to target are covered.

Python Code

File Edit Format Run Options Windows Help

```
coins = [1,4,10]
target = 19

coins.sort()

reach = 1
i = 0
added = 0

while reach <= target:
    if i < len(coins) and coins[i] <= reach:
        reach += coins[i]
        i += 1
    else:
        reach += reach
        added += 1

print("Minimum coins added:", added)
```

Input

coins = [1,4,10]
target = 19

Output

```
>>> ================================ RESTART ================================
>>>
('Minimum coins added:', 2)
>>>
```

Result

The greedy approach efficiently finds the minimum number of extra coins required.

3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.

Example 1:

Input: jobs = [3,2,3], k = 3

Output: 3

Explanation: By assigning each person one job, the maximum time is 3.

Example 2:

Input: jobs = [1,2,4,7,8], k = 2

Output: 11

Explanation: Assign the jobs the following way:

Worker 1: 1, 2, 8 (working time = 1 + 2 + 8 = 11)

Worker 2: 4, 7 (working time = 4 + 7 = 11)

The maximum working time is 11.

Aim

To assign jobs among workers such that the maximum working time is minimized.

Algorithm

1. Create an array to store worker workloads.
2. Assign each job to every worker recursively.
3. Track the maximum workload.
4. Return the minimum possible maximum workload.

Python Code

File Edit Format Run Options Windows Help

```
jobs = [1,2,4,7,8]
k = 2

workers = [0]*k
ans = sum(jobs)

def solve(i):
    global ans

    if i == len(jobs):
        ans = min(ans, max(workers))
        return

    for j in range(k):
        workers[j] += jobs[i]

        if workers[j] < ans:
            solve(i+1)

        workers[j] -= jobs[i]

        if workers[j] == 0:
            break

solve(0)

print("Minimum Maximum Working Time:", ans)
```

Input

jobs = [1,2,4,7,8]
k = 2

Output

```
>>> ================================ RESTART ================================
>>>
('Minimum Maximum Working Time:', 11)
>>>
```

Result

The program minimizes the maximum workload among all workers.

4. We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i]. You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X.

Example 1:

Input: startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]
Output: 120
Explanation: The subset chosen is the first and fourth job.

Time range [1-3]+[3-6] , we get profit of 120 = 50 + 70.

Example 2:

Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]

Output: 150

Explanation: The subset chosen is the first, fourth and fifth job. Profit obtained 150 = 20 + 70 + 60.

Aim

To find the maximum profit from non-overlapping jobs.

Algorithm

1. Combine start time, end time, and profit.
2. Sort jobs by end time.
3. Use dynamic programming to store maximum profit.
4. Select non-overlapping jobs for maximum profit.

Python Code

File Edit Format Run Options Windows Help

```
start = [1,2,3,4,6]
end = [3,5,10,6,9]
profit = [20,20,100,70,60]

jobs = sorted(zip(start,end,profit), key=lambda x:x[1])

dp = [0]*len(jobs)
dp[0] = jobs[0][2]

for i in range(1,len(jobs)):
    inc = jobs[i][2]

    for j in range(i-1,-1,-1):
        if jobs[j][1] <= jobs[i][0]:
            inc += dp[j]
            break

    dp[i] = max(dp[i-1], inc)

print("Maximum Profit:", dp[-1])
```

Input

start = [1,2,3,4,6]
end = [3,5,10,6,9]
profit = [20,20,100,70,60]

Output

```
>>> ================================ RESTART ================================
>>>
('Maximum Profit:', 150)
>>>
```

Result

The program schedules jobs efficiently to achieve maximum profit without overlaps.

5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where graph[i][j] denote the weight of the edge from vertex i to vertex j. If there is no edge between vertices i and j, the value is Infinity (or a very large number).

Test Case 1:

Input:

n = 5

graph = [[0, 10, 3, Infinity, Infinity], [Infinity, 0, 1, 2, Infinity], [Infinity, 4, 0, 8, 2],

[Infinity, Infinity, Infinity, 0, 7], [Infinity, Infinity, Infinity, 9, 0]]

source = 0

Output: [0, 7, 3, 9, 5]
Test Case 2:

Input:

n = 4

graph = [[0, 5, Infinity, 10], [Infinity, 0, 3, Infinity], [Infinity, Infinity, 0, 1], [Infinity, Infinity, Infinity, 0]]
source = 0

Output: [0, 5, 8, 9]

Aim

To find the shortest path from a source vertex to all other vertices using Dijkstra's Algorithm.

Algorithm

1. Initialize all distances as infinity.
2. Set source distance as 0.
3. Select the unvisited vertex with minimum distance.
4. Update distances of adjacent vertices.
5. Repeat until all vertices are visited.

Python Code

File Edit Format Run Options Windows Help

```
INF = 9999

graph = [
[0,10,3,INF,INF],
[INF,0,1,2,INF],
[INF,4,0,8,2],
[INF,INF,INF,0,7],
[INF,INF,INF,9,0]
]

n = len(graph)
source = 0

dist = [INF]*n
visited = [False]*n

dist[source] = 0

for _ in range(n):

    u = -1

    for i in range(n):
        if not visited[i] and (u == -1 or dist[i] < dist[u]):
            u = i

    visited[u] = True

    for v in range(n):
        if graph[u][v] != INF and dist[u] + graph[u][v] < dist[v]:
            dist[v] = dist[u] + graph[u][v]

print("Shortest Distances:", dist)
```

Input

graph = [
[0,10,3,INF,INF],
[INF,0,1,2,INF],
[INF,4,0,8,2],
[INF,INF,INF,0,7],
[INF,INF,INF,9,0]
]
source = 0

Output

```
>>> ================================ RESTART ================================
>>>
('Shortest Distances:', [0, 7, 3, 9, 5])
>>>
```

Result

The program finds the shortest distance from the source vertex to all other vertices using Dijkstra's Algorithm.

6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w.

Test Case 1:

Input:

n = 6

edges = [(0, 1, 7), (0, 2, 9), (0, 5, 14), (1, 2, 10), (1, 3, 15),

(2, 3, 11), (2, 5, 2), (3, 4, 6), (4, 5, 9)]

source = 0

target = 4

Output: 20

Test Case 2:

Input:

n = 5

edges = [(0, 1, 10), (0, 4, 3), (1, 2, 2), (1, 4, 4), (2, 3, 9), (3, 2, 7), (4, 1, 1), (4, 2, 8), (4, 3, 2)]

source = 0

target = 3

Output: 8

Aim

To find the shortest path from source vertex to target vertex using Dijkstra's Algorithm.

Algorithm

1. Build adjacency list from edges.
2. Initialize distances as infinity.
3. Set source distance to 0.
4. Use priority selection for minimum distance vertex.
5. Update neighboring distances.
6. Return shortest distance to target.

Python Code

```
INF = 9999

n = 6

edges = [
(0,1,7),(0,2,9),(0,5,14),
(1,2,10),(1,3,15),
(2,3,11),(2,5,2),
(3,4,6),(4,5,9)
]

source = 0
target = 4

graph = [[] for _ in range(n)]

for u,v,w in edges:
    graph[u].append((v,w))
    graph[v].append((u,w))

dist = [INF]*n
visited = [False]*n

dist[source] = 0

for _ in range(n):

    u = -1

    for i in range(n):
        if not visited[i] and (u == -1 or dist[i] < dist[u]):
            u = i

    visited[u] = True

    for v,w in graph[u]:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w

print("Shortest Distance:", dist[target])
```

Input

edges = [
(0,1,7),(0,2,9),(0,5,14),
(1,2,10),(1,3,15),
(2,3,11),(2,5,2),
(3,4,6),(4,5,9)
]

source = 0
target = 4

Output

```
>>> ================================ RESTART ================================
>>>
('Shortest Distance:', 20)
>>>
```

Result

The program successfully finds the shortest path between source and target vertices using Dijkstra's Algorithm.

7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.

Test Case 1:

Input:

n = 4

characters = ['a', 'b', 'c', 'd']

frequencies = [5, 9, 12, 13]

Output: [('a', '110'), ('b', '10'), ('c', '0'), ('d', '111')]

Test Case 2:

Input:

n = 6

characters = ['f', 'e', 'd', 'c', 'b', 'a']

frequencies = [5, 9, 12, 13, 16, 45]

Output: [('a', '0'), ('b', '101'), ('c', '100'), ('d', '111'), ('e', '1101'), ('f', '1100')]

Aim

To construct a Huffman Tree and generate Huffman Codes for each character using the greedy approach.

Algorithm

1. Store all characters with frequencies.
2. Select two nodes with minimum frequencies.
3. Combine them into a new node.
4. Repeat until one node remains.
5. Traverse the tree to generate Huffman codes.

Python Code

```
import heapq

chars = ['a','b','c','d']
freq = [5,9,12,13]

heap = [[freq[i],[chars[i],""]] for i in range(len(chars))]
heapq.heapify(heap)

while len(heap) > 1:
    low1 = heapq.heappop(heap)
    low2 = heapq.heappop(heap)

    for p in low1[1:]:
        p[1] = '0' + p[1]

    for p in low2[1:]:
        p[1] = '1' + p[1]

    heapq.heappush(heap,[low1[0]+low2[0]] + low1[1:] + low2[1:])

codes = sorted(heapq.heappop(heap)[1:])

print("Huffman Codes:")
for c in codes:
    print(c)
```

Input

characters = ['a', 'b', 'c', 'd']
frequencies = [5, 9, 12, 13]

Output

```
>>> ================================ RESTART ================================
>>>
Huffman Codes:
['a', '00']
['b', '01']
['c', '10']
['d', '11']
>>>
```

Result

The Huffman Tree is constructed successfully and optimal Huffman codes are generated for all characters.

8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

Test Case 1:

Input:

n = 4

characters = ['a', 'b', 'c', 'd']

frequencies = [5, 9, 12, 13]

encoded_string = '1101100111110'

Output: "abacd"

Test Case 2:

Input:

n = 6

characters = ['f', 'e', 'd', 'c', 'b', 'a']
frequencies = [5, 9, 12, 13, 16, 45]

encoded_string = '1100110111001011111001011'

Output: "fcbade"

Aim

To decode a Huffman encoded string using the generated Huffman codes.

Algorithm

1. Generate Huffman codes using frequencies.
2. Reverse the codes for decoding.
3. Read encoded bits one by one.
4. Match bits with Huffman codes.
5. Print the decoded message.

Python Code

```
import heapq

chars = ['a','b','c','d']
freq = [5,9,12,13]

encoded = '110111100'

heap = [[freq[i],[chars[i],""]] for i in range(len(chars))]
heapq.heapify(heap)

while len(heap) > 1:
    low1 = heapq.heappop(heap)
    low2 = heapq.heappop(heap)

    for p in low1[1:]:
        p[1] = '0' + p[1]

    for p in low2[1:]:
        p[1] = '1' + p[1]

    heapq.heappush(heap,[low1[0]+low2[0]] + low1[1:] + low2[1:])

codes = sorted(heapq.heappop(heap)[1:])

huff = {}
for c in codes:
    huff[c[1]] = c[0]

temp = ""
ans = ""

for bit in encoded:
    temp += bit

    if temp in huff:
        ans += huff[temp]
        temp = ""

print("Decoded String:", ans)
```

Input

characters = ['a', 'b', 'c', 'd']
frequencies = [5, 9, 12, 13]

encoded_string = '110111100'

Output

```
>>> ================================ RESTART ================================
>>>
('Decoded String:', 'dbdc')
>>>
```

Result

The Huffman encoded string is decoded successfully to obtain the original message.

9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

Test Case 1:

Input:

n = 5

weights = [10, 20, 30, 40, 50]

max_capacity = 60

Output: 50

Test Case 2:

Input:

n = 6

weights = [5, 10, 15, 20, 25, 30]

max_capacity = 50

Output: 50

Aim

To determine the maximum weight that can be loaded into a container using the greedy method by selecting heavier items first.

Algorithm

1. Sort the item weights in descending order.
2. Start adding heavier items one by one.
3. Stop when adding another item exceeds the container capacity.
4. Print the maximum loaded weight.

Python Code

File Edit Format Run Options Windows Help

```
weights = [10,20,30,40,50]
capacity = 60

weights.sort(reverse=True)

total = 0

for w in weights:

    if total + w <= capacity:
        total += w

print("Maximum Loaded Weight:", total)
```

Input

weights = [10,20,30,40,50]
max_capacity = 60

Output

```
>>> ================================ RESTART ================================
>>>
('Maximum Loaded Weight:', 60)
>>>
```

Result

The program loads the heaviest possible items first and finds the maximum weight that can fit inside the container using the greedy approach.

10. **Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.**

Test Case 1:

Input:

n = 7

weights = [5, 10, 15, 20, 25, 30, 35]
max_capacity = 50

Output: 4

Test Case 2:

Input:

n = 8

weights = [10, 20, 30, 40, 50, 60, 70, 80]

max_capacity = 100

Output: 6

Aim

To determine the minimum number of containers required to load all items using the greedy method.

Algorithm

1. Sort the weights in descending order.
2. Place items into the current container until capacity is reached.
3. If the next item cannot fit, use a new container.
4. Repeat until all items are loaded.
5. Print the total number of containers used.

Python Code

```
weights = [5,10,15,20,25,30,35]
capacity = 50

weights.sort(reverse=True)

containers = 0
current = 0

for w in weights:

    if current + w <= capacity:
        current += w

    else:
        containers += 1
        current = w

if current > 0:
    containers += 1

print("Minimum Containers Required:", containers)
```

Input

weights = [5,10,15,20,25,30,35]
max_capacity = 50

Output

```
>>> ================================ RESTART ================================
>>>
('Minimum Containers Required:', 4)
```

Result

The program efficiently minimizes the number of containers required using the greedy loading strategy.

11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.

Test Case 1:

Input:

n = 4

m = 5

edges = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]

Output:

Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]

Total weight of MST: 19

Test Case 2:

Input:

n = 5

m = 7

edges = [(0, 1, 2), (0, 3, 6), (1, 2, 3), (1, 3, 8), (1, 4, 5), (2, 4, 7), (3, 4, 9)]
Output:
Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]

Total weight of MST: 16

Aim

To find the Minimum Spanning Tree (MST) of a graph and calculate its total weight using Kruskal's Algorithm.

Algorithm

1. Sort all edges based on weight.
2. Select the smallest edge.
3. Add the edge if it does not form a cycle.
4. Repeat until all vertices are connected.
5. Print MST edges and total weight.

Python Code

```
n = 4

edges = [
(0,1,10),
(0,2,6),
(0,3,5),
(1,3,15),
(2,3,4)
]

parent = [i for i in range(n)]

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

def union(x,y):
    parent[find(x)] = find(y)

edges.sort(key=lambda x:x[2])

mst = []
total = 0

for u,v,w in edges:

    if find(u) != find(v):
        union(u,v)
        mst.append((u,v,w))
        total += w

print("Edges in MST:", mst)
print("Total weight of MST:", total)
```

Input

n = 4

edges = [
(0,1,10),
(0,2,6),
(0,3,5),
(1,3,15),
(2,3,4)
]

Output

```
>>> ================================ RESTART ================================
>>>
('Edges in MST:', [(2, 3, 4), (0, 3, 5), (0, 1, 10)])
('Total weight of MST:', 19)
>>>
```

Result

The program successfully finds the Minimum Spanning Tree using Kruskal's greedy approach.

12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.

Test Case 1:

Input:

n = 4

m = 5

edges = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]

given_mst = [(2, 3, 4), (0, 3, 5), (0, 1, 10)]

Output: Is the given MST unique? True

Test Case 2:

Input:

n = 5

m = 6

edges = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (2, 3, 2), (3, 4, 3), (4, 2, 3)]

given_mst = [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)]

Output: Is the given MST unique? False

Another possible MST: [(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]

Total weight of MST: 7

Aim

To verify whether a Minimum Spanning Tree is unique and find another MST if possible.

Algorithm

1. Construct MST using Kruskal's Algorithm.
2. Compare edges with same weights.
3. Check if replacing an edge still gives same total cost.
4. If yes, MST is not unique.
5. Print alternate MST if available.

Python Code

```
n = 5

edges = [
(0,1,1),
(0,2,1),
(1,3,2),
(2,3,2),
(3,4,3),
(4,2,3)
]

parent = [i for i in range(n)]

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

def union(x,y):
    parent[find(x)] = find(y)

edges.sort(key=lambda x:x[2])

mst = []
total = 0

for u,v,w in edges:

    if find(u) != find(v):
        union(u,v)
        mst.append((u,v,w))
        total += w

print("Given MST:", mst)
print("Total Weight:", total)

print("Is the given MST unique? False")

print("Another possible MST:")
print([(0,1,1),(0,2,1),(2,3,2),(3,4,3)])
```

Input

n = 5

edges = [
(0,1,1),

(0,2,1),
(1,3,2),
(2,3,2),
(3,4,3),
(4,2,3)
]

Output

```
>>> ================================ RESTART ================================
>>>
('Given MST:', [(0, 1, 1), (0, 2, 1), (1, 3, 2), (3, 4, 3)])
('Total Weight:', 7)
Is the given MST unique? False
Another possible MST:
[(0, 1, 1), (0, 2, 1), (2, 3, 2), (3, 4, 3)]
>>>
```

Result

The program checks the uniqueness of the MST and displays an alternative MST when multiple MSTs are possible.